

Nombre: Sergio Giovanni

Apellido: Alejo Bernuy

Código: 20200147

Fecha: 20/06

2. I. Análisis de arquitectura y concurrencia

1. Repositorio como SSOT: Función clave de centralizar y proveer datos (Encargado)

UI o viewModel siempre piden datos al repositorio

Garantizando que la app vea la misma información

Fachada

: Simplifica y oculta la complejidad del sistema de datos
Oculta en cierta parte la lógica de negocio

ROOM + RETROFIT

: UI solicita datos:

↳ Repositorio expone un flujo de datos desde la base de datos local (Room)

↳ Petición a una API remota (Retrofit)

↳ Respuesta de la red, el repositorio guarda los nuevos datos en ROOM

↳ UI observa ROOM, se actualiza automáticamente

La UI jamás se entera si el dato vino del servidor o almacenamiento local

2. viewModelScope: CoroutineScope para el ciclo de vida del ViewModel

Asegura la concurrencia estructurada (ciclo de vida de las operaciones asíncronas está atado al componente que las originó)

Gestión de memoria: Cancela automáticamente todas las coroutines que se estén ejecutando dentro de él cuando el ViewModel se destruye (se limpia de la memoria)
Evita fugas de memoria (impide que hilo en 2do plano intenten actualizar componentes que ya no existen)

Cerrar la pantalla: Promueve la destrucción del ViewModel, el viewModelScope se cancela de inmediato, cancela las peticiones de red del Retrofit (libera recursos y memoria antes de la llegada de la respuesta)

3. Flow (Cold / Frio): Reactivo bajo demanda

No emite datos hasta que haya un recolector activo (collect)

Cada recolector ejecuta el bloque de código desde el principio de manera independiente, sin almacenar estado

StateFlow (Hot / Caliente): Activo independientemente si hay suscriptores o no
Mantiene y emite el último estado conocido (requiere un valor inicial)
a cualquier nuevo recolector de manera inmediata

Preferencia en UI: Se prefiere Stateflow porque la interfaz de usuario en Android necesita un estado persistente y único

↳ Pantalla se rota, la actividad se recrea

↳ Flow dispara la petición o recarga dando error

↳ Stateflow: Entrega de inmediato el último estado guardado a la nueva vista sin repetir procesos

4. "DI manual": Se instancian objetos de infraestructura única (cliente red, Redis y repositorios) como propiedades dentro de una clase personalizada que hereda de Application. Componentes como la ViewModel acceden a estas dependencias a través del contexto de la aplicación (Simula un contenedor de dependencias centralizado sin librerías externas)

By Lazy: Delegado de inicialización perezosa

Serve para que el objeto no se cree en memoria cuando la aplicación arranca, sino estrictamente la primera vez que alguien lo solicita (lo usa)

3. II. Savoir de completar

1. El flujo de datos que emite automáticamente cada vez que una tabla de room cambie se implementa usando el tipo Flow

2. El operador Dispatchers.Default permite ejecutar tareas pesadas fuera del Main Thread, específicamente optimizado para tareas de CPU como el parseo masivo de datos

3. Para que un sensor en primer plano de ubicación sea válido en Android 10+, debe declararse en el manifiesto con el tipo Location

4. La función getApplicationContext() de la clase Application es la que garantiza que los componentes compartan la misma instancia del repositorio mediante un "cont"

5. El componente que permite transformar una API basada en callbacks antiguos en una función suspendible moderna se llama suspendCancellableCoroutine

6. El archivo de metadatos que describe la estructura de la app al sistema operativo se llama exactamente AndroidManifest.xml

7. En la arquitectura AOSP, la capa que actúa como puente estándar entre el framework y los controladores se llama HAL (Hardware Abstraction Layer)

8. El operador de Flow que permite combinar múltiples fuentes (Como GPS y Sensor) para emitir un estado unificado es: combine

4. II. Desarrollo de código y lógica personalizada

1. interfaz DAO

```
class UsuarioRepository (private val usuarioDao: UsuarioDao) { // lógica de negocio y acceso a
} // datos
class MyApp: Application () {
    // Inicialización perezosa de la Base de datos o DAO
    private val database: MyDatabase by lazy {
        MyDatabase.getDatabase (this)
    }
    // Inicialización perezosa del Repository pasando su dependencia (DAO)
    val usuarioRepository: UsuarioRepository by lazy {
        UsuarioRepository (database.usuarioDao())
    }
} // Simulación de la base de datos (funcional)
abstract class MyDatabase {
    abstract fun usuarioDao(): UsuarioDao
    companion object {
        fun getDatabase (context: Context): MyDatabase = throw Exception ("Simulado")
    }
}
```


2. Lógica Dinámica de Identidad

```
import kotlin.system.exitProcess
```

```
data class Usuario(  
    val nombre: String?,  
    val apellidos: String?,  
    val edad: Int?  
)
```

```
fun main() {  
    val miUsuario = Usuario(  
        nombre = "Sergio Giovanni",  
        apellidos = "Alejo Berny",  
        edad = 24  
    )
```

```
// Lógica de puntaje de Identidad usando operador Elvis  
val letrasNombre = miUsuario.nombre?.replace(" ", "")?.length ?: 0  
val letrasApellidos = miUsuario.apellidos?.replace(" ", "")?.length ?: 0
```

```
val puntajeIdentidad = letrasNombre + letrasApellidos
```

```
val nombreCompleto = "${miUsuario.nombre?.replace(" ", "") ?: ""} ${miUsuario.apellidos?.replace(" ", "") ?: ""}.trim()
```

```
println("Usuario: [$nombreCompleto], su puntaje de identidad es: [$puntajeIdentidad]")
```


5. IV. Arquitectura del AOSP Completo

1. Aplicaciones del sistema / Apps
2. Java API Framework
3. Android Runtime ART + Librerías Nativas
4. Hardware Abstraction Layer HAL
5. Linux Kernel

1. Aplicación del sistema / Apps:

Contiene las aplicaciones que usa el usuario, como teléfono, contactos, cámara, navegador y apps instaladas.

2. Java API Framework:

Proporciona las APIs que usan las apps para acceder a recursos del sistema.

3. Android Runtime ART + Librerías Nativas:

Ejecuta aplicaciones Android y contiene librerías base del sistema.

4. HAL: Actúa como puente estándar entre el framework de Android y los controladores de hardware.

5. Linux Kernel: Base del sistema. Gestiona memoria, proceso, seguridad, red, energía y drivers del hardware.

→ Componentes de Android Runtime ART

ART Android Runtime: Ejecuta el código de las aplicaciones Android.

Core Libraries: Provee clases fundamentales de Java/Kotlin usadas por las apps.

Garbage Collector: Administra automáticamente la memoria y libera objetos no utilizados.

Compilación AOT/JIT: Optimiza ejecución de las apps mediante compilación anticipada y en tiempo de ejecución.